

Shors_algo

September 17, 2026

Week-02:Shors Algorithm(Day 06)

```
[20]: import qiskit
import numpy as np

from math import gcd
from fractions import Fraction

from qiskit import QuantumCircuit, transpile
from qiskit.circuit.library import QFT, UnitaryGate
from qiskit_aer import AerSimulator

# =====
# SHOR'S ALGORITHM
# Factor N = 15
# =====

N = 15
a = 2

n_count = 4
n_work = 4

print("Qiskit version:", qiskit.__version__)
print("Factoring N =", N)
print("Chosen a =", a)

# =====
# Create modular multiplication matrix
#
#  $U|y\rangle = |a*y \bmod N\rangle$ 
# =====

def modular_multiplication(a, N, n):
```

```

dimension = 2 ** n

U = np.zeros(
    (dimension, dimension),
    dtype=complex
)

for y in range(dimension):

    if y < N:
        result = (a * y) % N
    else:
        result = y

    U[result, y] = 1

return U

# =====
# Create U for multiplication by 2 mod 15
# =====

U = modular_multiplication(
    a,
    N,
    n_work
)

U_gate = UnitaryGate(
    U,
    label="Multiply by 2 mod 15"
)

# =====
# Create quantum circuit
# =====

qc = QuantumCircuit(
    n_count + n_work,
    n_count
)

counting = list(range(n_count))
work = list(range(n_count, n_count + n_work))

```

```

# =====
# STEP 1: Hadamard gates
# =====

qc.h(counting)

# =====
# STEP 2: Prepare  $|1\rangle$ 
# =====

qc.x(work[0])

# =====
# STEP 3: Controlled modular exponentiation
# =====

for j in range(n_count):

    power = 2 ** j

    U_power = np.linalg.matrix_power(
        U,
        power
    )

    gate = UnitaryGate(
        U_power,
        label=f"U{power}"
    )

    controlled_gate = gate.control(
        1
    )

    qc.append(
        controlled_gate,
        [counting[j]] + work
    )

# =====
# STEP 4: Inverse QFT
# =====

```

```

iqft = QFT(
    n_count,
    inverse=True,
    do_swaps=True
)

qc.compose(
    iqft,
    qubits=counting,
    inplace=True
)

# =====
# STEP 5: Measurement
# =====

qc.measure(
    counting,
    range(n_count)
)

# =====
# Display circuit
# =====

print("\nQuantum Circuit:\n")

print(
    qc.draw(
        output="text"
    )
)

# =====
# SIMULATION
# =====

simulator = AerSimulator()

compiled_circuit = transpile(
    qc,
    simulator
)

```

```

result = simulator.run(
    compiled_circuit,
    shots=4096
).result()

counts = result.get_counts()

print("\nMeasurement results:")

for state, count in sorted(
    counts.items()
):

    print(
        state,
        ":",
        count
    )

# =====
# CLASSICAL POST-PROCESSING
# =====

print("\nPost-processing...\n")

for state, count in sorted(
    counts.items(),
    key=lambda x: x[1],
    reverse=True
):

    measurement = int(
        state,
        2
    )

    phase = measurement / (
        2 ** n_count
    )

    fraction = Fraction(
        phase
    ).limit_denominator(N)

    r = fraction.denominator

```

```

print(
    "State =", state,
    " Count =", count,
    " Phase =", round(phase, 4),
    " Candidate r =", r
)

# Period must be even
if r == 0 or r % 2 != 0:
    continue

# Calculate  $a^{(r/2)} \bmod N$ 
x = pow(
    a,
    r // 2,
    N
)

# Trivial cases
if x == 1 or x == N - 1:
    continue

# Calculate factors
p = gcd(
    x - 1,
    N
)

q = gcd(
    x + 1,
    N
)

if p not in (1, N) and q not in (1, N):

    print("\n=====")
    print("      SHOR ALGORITHM RESULT")
    print("=====")

    print("N =", N)
    print("a =", a)
    print("Period r =", r)
    print(" $a^{(r/2)} \bmod N$  =", x)

    print("\nFactors:")

```

```

print(
    N,
    "=",
    p,
    "x",
    q
)

break

```

Qiskit version: 2.4.1

Factoring N = 15

Chosen a = 2

/tmp/ipykernel_111939/2210997881.py:131: DeprecationWarning: The class ``qiskit.circuit.library.basis\_change.qft.QFT`` is deprecated as of Qiskit 2.1. It will be removed in Qiskit 3.0. ('Use qiskit.circuit.library.QFTGate or qiskit.synthesis.qft.synth_qft_full instead, for access to all previous arguments.',)

```
iqft = QFT(
```

Quantum Circuit:

```

q_0:  H                0      M
q_1:  H                1      M
q_2:  H                2      M
q_3:  H                3      M
q_4:  X  0      0      0      0
q_5:   1      1      1      1
      U^1    U^2    U^4    U^8
q_6:   2      2      2      2
q_7:   3      3      3      3

```

c: 4/

0 1 2 3

Measurement results:

0000 : 1023

0100 : 998

1000 : 1011

1100 : 1064

Post-processing...

State = 1100 Count = 1064 Phase = 0.75 Candidate r = 4

=====

SHOR ALGORITHM RESULT

=====

N = 15

a = 2

Period r = 4

$a^{(r/2)} \bmod N = 4$

Factors:

$15 = 3 \times 5$

[]: